

EXERCISE 11: TICKET ROUTING (INTENT AI)

The machine reads a ticket and sends it to the right team

AT A GLANCE

ML technique	NLP intent classification TF-IDF (words -> numbers) + Logistic Regression
Business problem	Read every incoming support ticket and auto-send it to the right team – auto triage - instantly, 24/7
Why this one	This is the MOST DEPLOYABLE exercise of the week. Cheap, fast, private, explainable.
Data	~245 synthetic Omantel tickets across 4 teams, generated
The lesson	In Exercise 2 the transformer won. Here the SIMPLE model wins because the job is different. Match the tool.

PRE CHECKLIST

- **Python notebook environment: ANACONDA NAVIGATOR → launch Jupyter Notebook**

OPENING THE EXERCISE

We saw a big deep-learning transformer beat our simple word-counter at reading feelings. You might be thinking: so, the fancy model always wins, right? Wrong. And this exercise is where we prove it."

"In Exercise 5 the machine read the FEELING/SENTIMENT in a message. Now it reads the MEANING what is this customer actually asking for? Every ticket that arrives at Omantel Support Mailbox has to go somewhere: billing, network, SIM activation, plan change."

"We're going to build the machine that reads a ticket and instantly sends it to the right team. And here for this job a simple, offline, transparent model is not just good enough. It is the RIGHT choice. This is the one you could actually deploy at Omantel on Monday."

STEP-BY-STEP WALKTHROUGH

Every code block below is copy-paste ready — paste the whole block into ONE notebook cell and run it (Shift + Enter).

STEP 1 - Load our tools

Cell 1. Three new sklearn tools join the usual crew: one to turn words into numbers, one to learn, one to split.

</> CODE - COPY & PASTE INTO ONE NOTEBOOK CELL

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

print("All tools loaded. Ready to build an auto-router.")
```

EXPECTED OUTPUT

All tools loaded. Ready to build an auto-router.

"One new name to notice: TfidfVectorizer. That is the tool that turns words into numbers. And LogisticRegression that is the exact same model we used on Day 1 to predict churn. Same Model, brand new job."

STEP 2 — Build the ticket database (4 teams)

Cell 2. We compose realistic tickets from fragments so no two are identical — just like real customers, who never phrase things the same way twice.

</> CODE - COPY & PASTE INTO ONE NOTEBOOK CELL

```
# Read the ticket database back from the CSV file
df = pd.read_csv("omantel_support_tickets.csv")
print(f"Loaded {len(df)} tickets from the CSV file.")
```

EXPECTED OUTPUT

Created 245 unique support tickets across 4 teams.

team	
Network	70
Billing	64
SIM_Activation	53
Plan_Change	58

STEP 3 — Look at the data

</> CODE - COPY & PASTE INTO ONE NOTEBOOK CELL

```
df.head(10)
```

EXPECTED OUTPUT

```
TKT70000 My new SIM is not activating please help SIM_Activation
TKT70001 I would like to switch to postpaid Plan_Change
TKT70003 The invoice is wrong can you explain Billing
TKT70005 What plans do you have for more data Plan_Change
...
```

"Read a couple out. A human sorts these without thinking — 'invoice' means billing, 'SIM' means activation. The question is whether the machine can learn that same instinct from examples. It can — and you'll see how simply."

STEP 4 Turn words into numbers (TF-IDF)

Cell 4. THE KEY CONCEPT. A machine cannot read 'bill'. TF-IDF converts each ticket into a row of numbers, where each number is how important a word is. Rare, meaningful words (invoice, signal, eSIM) score high; common words (the, my, is) score low.

</> CODE - COPY & PASTE INTO ONE NOTEBOOK CELL

```
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(df["ticket_text"])
y = df["team"]
print(f"Each ticket is now {X.shape[1]} number-columns the machine can read.")
```

EXPECTED OUTPUT

Each ticket is now 109 number-columns the machine can read.

"This is the moment text becomes maths. Every ticket is now a row of 124 numbers — one per word in our whole vocabulary. The word 'invoice' lights up for billing tickets; 'signal' lights up for network tickets."

"You do not need the formula behind TF-IDF. You need to know what it DOES: turns language into numbers, weighting words by how much they matter. That's the same bridge from yesterday — text to numbers — done in one line."

STEP 5 — Split: teaching set vs exam set

Cell 5. Exactly the same split idea as Exercise 1- the model studies 75%, and we test it on 25% it has never seen.

</> CODE - COPY & PASTE INTO ONE NOTEBOOK CELL

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)
print(f"Training tickets: {X_train.shape[0]} Exam tickets: {X_test.shape[0]}")
```

EXPECTED OUTPUT

Training tickets: 183 Exam tickets: 62

STEP 6 - Train the router

</> CODE - COPY & PASTE INTO ONE NOTEBOOK CELL

```
router = LogisticRegression(max_iter=1000)
router.fit(X_train, y_train)
print("Router trained.")
```

EXPECTED OUTPUT

Router trained.

Two lines again. Same LogisticRegression you met in Exercise 1 - the only thing that changed is we fed it words-turned-into-numbers instead of tenure and spend. Same tool, new job. That's the whole reward for learning the foundations well

STEP 7 - How good is it?

</> CODE - COPY & PASTE INTO ONE NOTEBOOK CELL

```
preds = router.predict(X_test)
acc = accuracy_score(y_test, preds) * 100
print(f"Routing accuracy: {acc:.1f}%")
print(classification_report(y_test, preds))
```

EXPECTED OUTPUT

```
Routing accuracy: 96.8%
              precision    recall  f1-score   support

   Billing      0.94      1.00      0.97        16
   Network      0.94      0.94      0.94        18
 Plan_Change    1.00      1.00      1.00        15
SIM_Activation  1.00      0.92      0.96        13

   accuracy                   0.97        62
  macro avg      0.97      0.97      0.97        62
 weighted avg    0.97      0.97      0.97        62
```

STEP 8- Confusion matrix: any mistakes?

</> CODE — COPY & PASTE INTO ONE NOTEBOOK CELL

```
labels = sorted(df["team"].unique())
cm = confusion_matrix(y_test, preds, labels=labels)
print(pd.DataFrame(cm, index=labels, columns=labels).to_string())
```

EXPECTED OUTPUT

	Billing	Network	Plan_Change	SIM_Activation
Billing	16	0	0	0
Network	1	17	0	0
Plan_Change	0	0	15	0
SIM_Activation	0	1	0	12

STEP 9 - The hard ones (the real lesson)

Cell 9. THE MOST IMPORTANT CELL OF THE EXERCISE. We route four brand-new tickets that each mention TWO topics — and watch the machine become unsure. This is where trainees learn what a confidence score is, and why humans stay in the loop.

</> CODE — COPY & PASTE INTO ONE NOTEBOOK CELL

```
hard = [
    "My new plan shows the wrong price on my bill",
    "I changed my plan but now the internet does not work",
    "The offer charged me but my SIM is not active",
    "Slow internet and my bill went up too",
]
probs = router.predict_proba(vectorizer.transform(hard))
classes = router.classes_
for t, pr in zip(hard, probs):
    top = sorted(zip(classes, pr), key=lambda x: -x[1])[1:2]
    print(f"{t}")
    print(f"  -> {top[0][0]} ({top[0][1]*100:.0f}%) | 2nd guess: {top[1][0]} ({top[1][1]*100:.0f}%)\\n")
```

EXPECTED OUTPUT

```
My new plan shows the wrong price on my bill
  -> Billing (53%) | 2nd guess: SIM_Activation (23%)

I changed my plan but now the internet does not work
  -> Network (50%) | 2nd guess: SIM_Activation (19%)

The offer charged me but my SIM is not active
  -> Network (31%) | 2nd guess: SIM_Activation (28%)

Slow internet and my bill went up too
  -> Network (40%) | 2nd guess: Billing (31%)
```

Look at row three. 31% versus 28%. That is not a confident model that is a shrug. And that shrug is valuable information. If Omantel deployed this exactly as-is with no safety net, this ticket has a real chance of going to the wrong team. But if you build in a rule say, 'anything under 80% confidence gets a human's eyes before it's routed' this exact ticket gets caught and reviewed, instead of silently misrouted

STEP 10 - Route live tickets

</> CODE - COPY & PASTE INTO ONE NOTEBOOK CELL

```
new_tickets = [
    "My bill is too high this month please check",
    "There is no network signal in Nizwa",
    "I cannot activate my new eSIM",
    "I want to upgrade to a bigger data plan",
]
routes = router.predict(vectorizer.transform(new_tickets))
print("LIVE ROUTING:")
for t, r in zip(new_tickets, routes):
    print(f"  -> [{r:15}] {t}")
```

EXPECTED OUTPUT

```
LIVE ROUTING:
-> [Billing      ] My bill is too high this month please check
-> [Network      ] There is no network signal in Nizwa
-> [SIM_Activation] I cannot activate my new eSIM
-> [Plan_Change  ] I want to upgrade to a bigger data plan
```

Simple router vs a GenAI/LLM: when to use which

```
# CONCEPT ONLY — no code to run. The honest comparison.
#
# YOUR ROUTER (today):      A GENAI / LLM:
# matches WORDS it saw      understands MEANING, even new wording
# fast & basically free      slower, costs money per ticket
# runs offline, private      needs internet + an API key
# fully explainable          a black box
# -> millions of routine      -> the weird, tricky, high-value
# tickets, cheaply            tickets only
#
# Most real systems use BOTH, in that order: the cheap router
# triages everything, the LLM handles what the router is unsure about.
```

"the transformer beat our word-counter at sentiment, because feeling is messy and worth the cost. But here, for routing millions of tickets, the SIMPLE model wins: it's cheap, fast, private, and you can see why it decided. Same week, opposite answer — because the JOB is different."

"That is the single most valuable skill you take from this program. Not knowing how to call an LLM — knowing WHEN to, and when a simpler tool is the smarter, more deployable choice. That is what makes you dangerous in the good way."